Chapter 1 solutions: ADTs, correctness, and cost

DS01-E01

Writing bottom to top: push(2) gives [2], success; push(2) gives [2,2], success; push(5) gives [2,2,5], success; pop gives [2,2] and returns 5; push(7) gives [2,2,7], success; push(8) gives [2,2,7], failure. A set loses both order and multiplicity, so it cannot determine the next pop.

DS01-E02

Allocate separate storage and leave the old state unchanged while copying the elements and constructing the new one. On failure, destroy only constructed new elements and release the new storage. Commit the new pointer, capacity, and size only after all construction succeeds; then destroy the old storage. Copying must not modify the source; a throwing move that modifies its source does not generally provide this guarantee. Commit and cleanup must not throw.

DS01-E03

(l,r,m)=(0,5,2): A[2] is not less, so r=2. Then (0,2,1): r=1. Then (0,1,0): A[0]<2, so l=1. Return 0-based 1. FIRST-MATCH returns 1-based 2. If absent, FIRST-MATCH returns NIL, whereas lower_bound returns an insertion position, possibly n; first check i<n before reading A[i], then test both !comp(A[i],x) and !comp(x,A[i]) for equivalence.

DS01-E04

For i=1,…,6, the body counts are 7,4,3,2,2,2, summing to 20; the corresponding condition-test counts are 8,5,4,3,3,3, summing to 26. Each i has one final failed test. Since x=0 still satisfies the condition, each body count is floor(6/i)+1.

DS01-E05

One recursive call yields 8→4→2→1, or 4 calls, with $T(n)=T(n/2)+\Theta(1)=\Theta(\log n+1)$. Two calls produce 1+2+4+8=15 calls, with $T(n)=2T(n/2)+\Theta(1)=\Theta(n)$. These counts describe the specified source model; optimized machine code needs separate analysis.

DS01-E06

F(0),…,F(5)=0,1,1,2,3,5. The call recurrence is C(0)=C(1)=1 and C(n)=1+C(n−1)+C(n−2), yielding C(2),…,C(5)=3,5,9,15. The identity holds at n=0,1; substitution into the recurrence gives 1+[2F(n)−1]+[2F(n−1)−1]=2F(n+1)−1. G(0),…,G(5)=1,1,2,3,5,8. Its recursion tree has the same shape, so C(5)=15.

DS01-E07

Appends 4,7,13 copy 3,6,12 elements respectively, totaling 21 copies plus 13 new writes, or 34. At n=0 copies and writes are zero. For $n \geq 1$ without a resize, copies are $0<2n$. Otherwise let $c<n$ be the last old capacity: copies sum to $3+6+\ldots+c=2c-3<2n$. For initial capacity $c_0 \geq 1$ the same sum is $2c-c_0<2n$. Allocation and element-specific costs are excluded from this unit model.

DS01-E08

Before: $\Phi(4,4)=4$. After doubling to 8 and inserting: $\Phi(5,8)=2$. Actual cost is 4 copies plus one write, so amortized cost is $5+2-4=3$. Without resize the potential increases by at most two, giving amortized cost at most three. Summing cancels intermediate potential changes; nonnegative final potential and zero initial potential bound total actual cost by total amortized cost.

DS01-E09

0111→1000 flips four bits, with potential 3→1, so amortized cost is $4-2=2$. For 1111→0000, actual cost is 4 and potential change is −4, giving amortized cost 0. A sequence bound does not imply a per-operation actual-cost bound of two.

DS01-E10

The subproblems have sizes n,n−1,…,1 and comparisons total $(n-1)+\ldots+1=n(n-1)/2$. All pivot keys are equal, so random selection cannot change the split. Partition into less, equal, and greater regions and recurse only on less and greater; the all-equal case ends after the first partition.

DS01-E11

For n≥1, 3n+8≤11n, so c=11 and n_0=1 suffice. If 4^n∈O(2^n), then 2^n would be bounded by a constant for all sufficiently large n, a contradiction. Worst case maximizes over legal inputs; expectation requires a specified probability distribution; amortization bounds a sequence’ s total cost and allocates it among operations.

DS01-E12

Worst-case comparisons are Θ(log(n+1)); with at most O(L) character operations per comparison the total is O(L log(n+1)). Including indexing, length checks and control, total time is O((L+1)(1+log(n+1))). For n=0, the insertion position is 0 and no elements are compared, but constant control cost remains. Long common prefixes can require Θ(L) character operations for one comparison, so comparison is not constant time when L varies.

DS01-E13

Bitwise addition processes at most L bits per operation, including carry and sign handling, for $O(nL)$ total bit operations; the empty sequence only initializes the accumulator. Each input 2^N and -2^N needs $\Theta(N)$ bits even though the result is zero, so reading or bitwise processing can take $\Omega(N)$ time. If operands fit in fixed-size machine words, operations do not overflow, and the word-RAM model applies, each addition costs $O(1)$ and the total is $O(n)$.

References and attribution

Lin Cing, handwritten notes on data structures.
Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study